

Guía de trabajo — Práctica 02

Sistema de Archivos vs. Bases de Datos · Equipo de 5 integrantes

Esta guía propone una distribución equitativa del trabajo para desarrollar la Práctica 02 de Fundamentos de Bases de Datos. Se basa en los requisitos de la práctica: análisis de requerimientos, prototipo para PuellaGame, almacenamiento en CSV, CRUD de sucursales/premios/clientes, consultas por llave, validaciones, manejo de excepciones, documentación y reporte final.

Objetivo de la distribución: que cada integrante tenga una responsabilidad clara de código, documentación y pruebas, evitando que una sola persona cargue con toda la integración o el reporte.

1. Resumen de responsabilidades

Integrante	Responsabilidad	Entregables principales
1	Arquitectura, análisis y CSV	Diseño común, entidades/llaves, manejo de archivos CSV, parte del reporte.
2	Sucursal	Clase y CRUD de Sucursal, validaciones, pruebas y documentación.
3	Premio	Clase y CRUD de Premio, validaciones, pruebas y documentación.
4	Cliente	Clase y CRUD de Cliente, validaciones, pruebas y documentación.
5	Menú e integración	Menú, excepciones, integración, pruebas generales, README y apoyo al PDF.

2. Antes de programar: trabajo de los cinco

- **Leer la práctica completa.** Identificar todas las entregas y restricciones.
- **Definir las tres entidades:** Sucursal, Premio y Cliente.
- **Definir atributos y llaves.** Acordar nombres y tipos de datos antes de programar.
- **Definir el formato de los CSV.** El orden de columnas debe ser idéntico para todos.
- **Definir una convención común.** Nombres de clases, métodos, excepciones y mensajes.
- **Definir la estructura del proyecto.** Acordar dónde estarán SRC, CSV y Doc según los lineamientos de entrega.

Hint de integración

No conviene que cada persona empiece a programar sin haber acordado el formato de datos. Si una clase interpreta las columnas del CSV de una forma y otra clase las interpreta de otra, la integración se complicará al final.

3.1. Integrante 1 — Arquitectura, análisis y manejo de archivos

Objetivo: establecer la estructura común para que los otros cuatro integrantes puedan trabajar sin contradicciones.

Pasos

1. Analizar los requerimientos del caso de uso y separar requerimientos funcionales y no funcionales.
2. Ayudar a definir las entidades Sucursal, Premio y Cliente, sus atributos y sus llaves.
3. Definir el formato de cada CSV: nombre del archivo, encabezados y orden de columnas.
4. Diseñar la estructura general del programa y las responsabilidades de las clases.
5. Implementar la utilidad o clase común para leer y escribir archivos CSV.
6. Considerar qué sucede si un archivo no existe, está vacío o contiene información inválida.

7. Redactar la parte del reporte correspondiente al análisis de requerimientos.

Hints

- | |
|---|
| • Intentar que el manejo de archivos sea reutilizable por las tres entidades. |
| • No mezclar toda la lógica del menú con la lectura/escritura de CSV. |
| • Acordar con el equipo los tipos de datos antes de implementar las validaciones. |
| • Documentar cada clase y método que implemente. |

3.2. Integrante 2 — Sucursal

Objetivo: implementar de principio a fin el manejo de la entidad Sucursal.

Pasos

1. Crear la clase Sucursal con los atributos acordados por el equipo.
2. Implementar Agregar: capturar datos, validar y guardar en el CSV.
3. Implementar Consultar: recibir la llave y devolver todos los datos de la sucursal.
4. Implementar Editar: localizar por llave, modificar y persistir los cambios.
5. Implementar Eliminar: localizar por llave, eliminar y persistir los cambios.
6. Validar datos y controlar llaves duplicadas o registros inexistentes.
7. Realizar pruebas normales y de error.
8. Documentar la clase y todos sus métodos.

Hints

- | |
|--|
| • Usar la misma convención de CRUD que Premio y Cliente. |
| • Probar consultar, editar y eliminar una llave que no existe. |
| • Comprobar que los cambios sobreviven después de cerrar y volver a abrir el programa. |
| • Evitar duplicar lógica de CSV si el integrante 1 ya creó una utilidad común. |

3.3. Integrante 3 — Premio

Objetivo: implementar de principio a fin el manejo de la entidad Premio.

Pasos

1. Crear la clase Premio con los atributos acordados.
2. Implementar Agregar, incluyendo validaciones.
3. Implementar Consultar por llave y devolver todos los datos.
4. Implementar Editar y persistir los cambios en el CSV.
5. Implementar Eliminar y persistir los cambios.
6. Validar especialmente los campos que deban almacenar números.
7. Probar casos válidos, inválidos, duplicados e inexistentes.
8. Documentar la clase y todos sus métodos.

Hints

- | |
|--|
| • No aceptar texto en campos que deben ser numéricos. |
| • Probar entradas como texto donde se espera un número. |
| • No asumir que un registro siempre existe. |
| • Mantener el mismo formato de CSV y convenciones que las demás entidades. |

3.4. Integrante 4 — Cliente

Objetivo: implementar de principio a fin el manejo de la entidad Cliente.

Pasos

1. Crear la clase Cliente con los atributos acordados.
2. Implementar Agregar con validación y persistencia.
3. Implementar Consultar utilizando la llave de la entidad.
4. Implementar Editar y actualizar el CSV.
5. Implementar Eliminar y actualizar el CSV.
6. Validar tipos de datos, llaves y registros inexistentes.
7. Realizar pruebas de funcionamiento y de errores.
8. Documentar la clase y todos sus métodos.

Hints

- | |
|--|
| • Mantener la lógica consistente con Sucursal y Premio. |
| • Comprobar que la consulta devuelva todos los atributos asociados al cliente. |
| • Probar qué ocurre si el usuario introduce datos vacíos o tipos incorrectos. |
| • Verificar la persistencia cerrando y abriendo de nuevo el programa. |

3.5. Integrante 5 — Menú, integración, excepciones y pruebas

Objetivo: convertir los módulos individuales en un programa completo y robusto.

Pasos

1. Diseñar el menú principal y los submenús de Sucursal, Premio y Cliente.
2. Conectar cada opción del menú con el CRUD correspondiente.
3. Implementar o coordinar el manejo de excepciones para entradas inválidas y fallos de ejecución.
4. Integrar las clases desarrolladas por los demás integrantes.
5. Realizar pruebas de extremo a extremo de las tres entidades.
6. Preparar el README y coordinar la revisión final del PDF.
7. Verificar que la estructura final de entrega cumpla con los lineamientos.

Hints

- | |
|---|
| • El menú debe encargarse principalmente de la interacción y llamar a métodos de las clases. |
| • Los mensajes de error deben ayudar al usuario a entender qué salió mal. |
| • No esperar hasta el último día para integrar: hacer pequeñas integraciones durante el desarrollo. |

- Probar opciones inválidas, entradas no numéricas, llaves inexistentes y problemas con archivos.

4. CRUD que debe cumplir cada entidad

Cada una de las tres entidades debe tener las cuatro operaciones solicitadas por la práctica. La consulta debe recibir la llave de la entidad y devolver todos sus datos relacionados.

Operación	Qué debe ocurrir	Casos que probar
Agregar	Capturar datos, validar y guardar en CSV.	Datos válidos, llave duplicada, tipo incorrecto.
Consultar	Recibir la llave y devolver todos los datos.	Llave existente e inexistente.
Editar	Localizar por llave, modificar y persistir.	Registro existente, inexistente y datos inválidos.
Eliminar	Localizar por llave, eliminar y persistir.	Registro existente e inexistente.

5. Validaciones y manejo de errores

- Campos numéricos: aceptar únicamente valores numéricos donde corresponda.
- Llaves: evitar duplicados al agregar.
- Consultas: informar cuando no exista la llave buscada.
- Ediciones y eliminaciones: comprobar primero que el registro exista.
- Menú: manejar opciones fuera del rango esperado.
- Archivos: contemplar problemas de lectura/escritura y archivos inexistentes.
- Excepciones: informar al usuario de manera clara cuando algo salga mal.

6. Plan de pruebas del equipo

Prueba	Esperado
Agregar Sucursal válida	Se guarda en su CSV.
Consultar Sucursal existente	Devuelve todos sus datos.
Consultar Sucursal inexistente	Mensaje de error/controlado.
Editar Sucursal	El CSV refleja los cambios.
Eliminar Sucursal	El registro deja de existir.
Repetir las 5 pruebas anteriores con Premio	Mismo comportamiento esperado.
Repetir las 5 pruebas anteriores con Cliente	Mismo comportamiento esperado.
Texto en campo numérico	El programa no falla y solicita un valor válido.
Opción inválida del menú	Mensaje controlado.
Cerrar y abrir el programa	Los datos previamente guardados permanecen.

7. División del reporte PDF

Integrante	Sección sugerida
1	Análisis de requerimientos: requerimientos candidato, contexto, funcionales y no funcionales.
2	Descripción y funcionamiento de Sucursal; pruebas relevantes.
3	Descripción y funcionamiento de Premio; pruebas relevantes.
4	Descripción y funcionamiento de Cliente; pruebas relevantes.
5	Comparación sistema de archivos vs. base de datos, conclusión, README y unificación del documento.

8. Orden recomendado de trabajo

- **Etapas 1 — Los cinco juntos:** leer la práctica, definir entidades, atributos, llaves, CSV y estructura.
- **Etapas 2 — Desarrollo:** cada integrante trabaja en su responsabilidad, manteniendo las convenciones acordadas.
- **Etapas 3 — Integración temprana:** conectar módulos progresivamente; no dejar toda la integración para el final.
- **Etapas 4 — Pruebas:** los cinco prueban el programa completo, no solamente su módulo.
- **Etapas 5 — Documentación:** cada integrante documenta sus clases y métodos.
- **Etapas 6 — Entrega:** revisar código, CSV, documentación, Practica02.pdf, README y estructura del ZIP.

9. Checklist final

- Sucursal: agregar, consultar, editar y eliminar.
- Premio: agregar, consultar, editar y eliminar.
- Cliente: agregar, consultar, editar y eliminar.
- Los datos persisten en archivos CSV.
- Las consultas utilizan la llave de la entidad.
- Los campos numéricos validan que se introduzcan números.
- Existen mensajes/control de errores y manejo de excepciones.
- Existe al menos un menú de interacción.
- Cada clase está documentada.
- Cada método implementado está documentado.
- El reporte incluye el análisis del caso de uso.
- El reporte incluye 5 diferencias entre archivos y bases de datos.
- El reporte justifica qué alternativa es más conveniente.
- El README está preparado.
- El ZIP tiene la estructura exigida por los lineamientos.

10. Regla de oro para que sea equitativo

Cada integrante debe tener: (1) una responsabilidad concreta de código, (2) una sección del reporte, (3) pruebas de su módulo y (4) participación en la integración final. De esta forma, nadie queda únicamente haciendo documentación ni una sola persona queda responsable de juntar todo al final.

Fuente: Práctica 02 — Sistema de Archivos vs. Bases de Datos, Fundamentos de Bases de Datos, UNAM, 2027-I. Esta guía es una propuesta de organización del trabajo y no sustituye los lineamientos de entrega.